



Specifica Tecnica

02/04/2026



Revisioni

Ver.	Autore	Modifiche	Data	Verifica
0.6	Stefano Maso	Modifica sezione LLM	24/04/2026	-
0.5	Giuliano Banchieri	Stesura sezione architettura di deployment e organizzazione container	21/04/2026	Spadiliero Ruben
0.4	Stefano Maso	Aggiunte sottosezioni per l'importazione	10/04/2026	Besnik Murtezan
0.3	Stefano Maso	Stesura sezione 3.1.1.2 e sottosezioni	09/04/2026	Besnik Murtezan
0.2	Stefano Maso	Stesura sezione 3.1, 3.1.1 e sottosezioni relative all'architettura del backend e relativi diagrammi delle classi	08/04/2026	Besnik Murtezan
0.1	Giuliano Banchieri	Prima stesura: struttura del documento, tecnologie, riferimenti, introduzione	02/04/2026	Niccolò Feltrin



Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Scopo del prodotto	6
1.3	Riferimenti	6
1.3.1	Riferimenti normativi	6
1.3.2	Riferimenti informativi	6
2	Tecnologie	8
2.1	Tecnologie di codifica	8
2.1.1	TypeScript	8
2.1.2	Vue.js	8
2.1.3	Marked.js	8
2.1.4	CSS	8
2.1.5	Tailwind CSS	8
2.1.6	HTML	9
2.1.7	PHP	9
2.1.8	Laravel	9
2.2	Tecnologie di verifica	9
2.2.1	PHPUnit	9
2.3	Tecnologie per la persistenza dei dati (OPT)	9
2.3.1	PostgreSQL	9
2.4	Tecnologie di supporto	9
2.4.1	Vite	10
2.4.2	Composer	10
2.4.3	nginx	10
2.4.4	PHP-FPM	10
2.4.5	Docker	10
2.4.6	Docker Compose	10
2.5	Modelli LLM esterni	10
3	Architettura	11
3.1	Architettura logica	11
3.1.1	Backend	11
3.1.1.1	Gestione note e Esportazione	12
3.1.1.1.1	NoteController	12
3.1.1.1.2	NoteService	12
3.1.1.1.3	NoteRepositoryInterface	13
3.1.1.1.4	NoteRepository	13
3.1.1.1.5	Note	13
3.1.1.1.6	ImportController	13
3.1.1.1.7	ImportService	14
3.1.1.1.8	ImportStrategyFactory	14
3.1.1.1.9	ImportStrategyInterface	14
3.1.1.1.10	MdImport	14
3.1.1.1.11	ExportController	14
3.1.1.1.12	ExportService	14
3.1.1.1.13	ExportStrategyFactory	15
3.1.1.1.14	ExportStrategyInterface	15
3.1.1.1.15	MdExport	15
3.1.1.1.16	HtmlExport	15
3.1.1.1.17	PdfExport	15
3.1.1.2	Interazione con LLM ^G	16
3.1.1.2.1	LlmController	16



3.1.1.2.2	LlmService	16
3.1.1.2.3	LlmStrategyFactory	17
3.1.1.2.4	LlmProcessorResponse	17
3.1.1.2.5	LlmStrategyInterface	17
3.1.1.2.6	LlmContext	17
3.1.1.2.7	Strategy concrete	17
3.2	Architettura di deployment	18
3.2.1	Organizzazione dei container	18
3.3	Architettura di dettaglio	18
4	Stato dei requisiti funzionali	19



Indice delle immagini

1	Diagramma delle classi per Gestione note - Esportazione - Importazione	12
2	Diagramma delle classi per Interazione con LLM	16



1 Introduzione

1.1 Scopo del documento

Il presente documento contiene informazioni dettagliate sulle tecnologie utilizzate e sulle scelte progettuali del gruppo Tool-8 per la realizzazione del prodotto **Second Brain** su richiesta dell'azienda **Zucchetti S.p.A.**

Sono fornite descrizioni delle tecnologie scelte per backend, frontend e di supporto, i diagrammi delle classi che rappresentano in modo dettagliato l'architettura logica e una tabella di tracciamento dei requisiti soddisfatti.

1.2 Scopo del prodotto

"Estendere il note-taking con i Large Language Models."

L'obiettivo del capitolato^G C6 **"Second Brain"** è lo sviluppo^G di un applicativo che fornisca all'utente un editor di file MD con funzionalità^G AI che semplifichino la scrittura tramite funzionalità^G come: riassunto, traduzione, critica, correzione, e generazione del testo.

L'utente avrà la possibilità di utilizzare queste funzionalità^G anche tramite interrogazioni in linguaggio naturale che verranno processate da un LLM^G contattato dal sistema.

Il software sarà sviluppato sotto forma di applicazione web dove l'utente potrà creare, salvare, eliminare e modificare le sue note.

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Capitolato^G C6 - Progetto^G Second Brain:
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento progetto^G didattico:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
- Norme di Progetto^G v1.1
- Analisi dei Requisiti^G v1.5

1.3.2 Riferimenti informativi

- Verbali
- Glossario v1.0
- Lezioni del corso di *Ingegneria del software*^G aggiornate al 02/04/2026:
 - *"Progettazione^G software (T6)"*:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>
 - *"Progettazione^G e programmazione: Diagrammi delle classi (UML)"*:
<https://www.math.unipd.it/~rcardin/swea/2023/Diagrammi%20delle%20Classi.pdf>
 - *"Analisi e descrizione delle funzionalità^G: Diagrammi delle attività (UML)"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Diagrammi%20di%20Attivit%C3%A0.pdf>
 - *"Progettazione^G: I pattern architetturali"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Software%20Architecture%20Patterns.pdf>
 - *"Progettazione^G: Il pattern Dependency Injection"*:
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Architetturali%20-%20Dependency%20Injection.pdf>



- "Progettazione^G: Pattern Architeturali per le applicazioni con UI":
<https://www.math.unipd.it/~rcardin/sweb/2022/L02.pdf>
- "Progettazione^G: i pattern creazionali (GoF)":
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Creazionali.pdf>
- "Progettazione^G: I pattern strutturali (GoF)":
<https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Strutturali.pdf>
- "Progettazione^G: I pattern di comportamento (GoF)":
https://drive.google.com/file/d/1cpi6r0RMxFtC91nI6_sPrG1Xn-28z8eI/view?usp=sharing
- "Programmazione: SOLID programming":
<https://drive.google.com/file/d/1o1Xun2dVVc3mDiaGyN0FrDJhho031fLQ/view?pli=1>



2 Tecnologie

In questa sezione sono descritte le tecnologie utilizzate per lo sviluppo^G di **Second Brain**. Sono inclusi gli strumenti, i framework^G e le librerie utilizzati per lo sviluppo^G di backend, frontend e della parte di persistenza dei dati ma anche quelli di supporto, di verifica^G e i servizi esterni.

2.1 Tecnologie di codifica

Framework^G e librerie utilizzate per lo sviluppo^G backend e frontend.

2.1.1 TypeScript

TypeScript è un linguaggio basato su JavaScript che introduce la tipizzazione statica, migliorando la manutenibilità e la robustezza del codice.

- **Versione:** 5.9.3
- **Documentazione:** <https://www.typescriptlang.org/docs/> (Consultato 09/04/2026)

2.1.2 Vue.js

Vue è un framework^G open source per lo sviluppo^G di interfacce utente e single-page-application. Si basa su HTML, CSS, e JavaScript e fornisce un modello di programmazione dichiarativa basato su componenti che aiuta l'utilizzatore a costruire interfacce reattive di qualsiasi complessità.

- **Versione:** 3.5+
- **Documentazione:** <https://vuejs.org/guide/introduction.html>

2.1.3 Marked.js

Marked.js è una libreria open-source scritta in JavaScript utilizzata per il parsing e la conversione di testo in formato Markdown. Consente di trasformare contenuti Markdown in HTML in modo rapido, supportando estensioni personalizzate e integrazione lato client e server.

- **Versione:**
- **Documentazione:** <https://marked.js.org/>

2.1.4 CSS

CSS (Cascading Style Sheets) è un linguaggio utilizzato per la formattazione di documenti HTML, XHTML e XML.

- **Versione:** CSS3
- **Documentazione:** <https://developer.mozilla.org/en-US/docs/Web/CSS>

2.1.5 Tailwind CSS

Tailwind CSS è un framework^G CSS open source. Offre una serie di classi CSS "di utilità". Queste classi vengono poi personalizzate e combinate secondo necessità per definire lo stile di ogni elemento.

- **Versione:** 4.2.2
- **Documentazione:** <https://tailwindcss.com/docs/installation/using-vite>



2.1.6 HTML

HTML (HyperText Markup Language) è il linguaggio di markup più utilizzato al mondo per i documenti web. È un linguaggio di pubblico dominio e le sue regole sono dettate da **W3C** (World Wide Web Consortium)

- **Versione:** HTML5
- **Documentazione:** <https://developer.mozilla.org/en-US/docs/Web/HTML>

2.1.7 PHP

PHP è un linguaggio di programmazione lato server progettato per lo sviluppo^G web, utilizzato per implementare la logica lato backend.

- **Versione:** 8.4.19
- **Documentazione:** <https://www.php.net/manual/it/>

2.1.8 Laravel

Laravel è un framework^G open source scritto in PHP utilizzato per lo sviluppo^G di applicazioni web moderne. Adotta il pattern architetturale MVC (Model View Controller) e fornisce strumenti integrati per la gestione del routing, dell'autenticazione e dell'accesso ai dati.

- **Versione:** 12.56.0
- **Documentazione:** <https://laravel.com/docs/13.x>

2.2 Tecnologie di verifica

Strumenti utilizzati per l'analisi del codice prodotto.

2.2.1 PHPUnit

PHPUnit è un framework^G utilizzato per l'analisi del codice PHP tramite test^G di unità. Lo scopo è trovare eventuali errori introdotti con modifiche al codice ed evitare che si verifichi una regressione.

- **Versione:** 11.5.55
- **Documentazione:** <https://phpunit.de/documentation.html>

2.3 Tecnologie per la persistenza dei dati (OPT)

2.3.1 PostgreSQL

PostgreSQL è un sistema di gestione di database relazionali open source compatibile con i sistemi operativi più diffusi.

- **Versione:** 17
- **Documentazione:** <https://www.postgresql.org/docs/>

2.4 Tecnologie di supporto

Strumenti utilizzati per facilitare lo sviluppo^G del prodotto da parte del gruppo.



2.4.1 Vite

Vite è uno strumento di sviluppo^G frontend che funge da dev server e build tool.

- **Versione:** 7.3.1
- **Documentazione:** <https://vite.dev/guide/>

2.4.2 Composer

Composer è un gestore di dipendenze per il linguaggio PHP utilizzato per installare, aggiornare e gestire automaticamente librerie e pacchetti necessari a un progetto.

- **Versione:** 2.9.5
- **Documentazione:** <https://getcomposer.org/doc/>

2.4.3 nginx

nginx è un web server open source leggero e ad alte prestazioni, utilizzabile anche come reverse proxy, load balancer, cache HTTP.

- **Versione:** 1.27
- **Documentazione:** <https://nginx.org/en/docs/>

2.4.4 PHP-FPM

- **Versione:** 8.4.19
- **Documentazione:** <https://www.php.net/manual/en/install.fpm.php>

2.4.5 Docker

Docker^G è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati e leggeri, detti container. Ogni container^G include tutte le dipendenze necessarie al funzionamento del software ed è gestito in modo indipendente rispetto agli altri.

- **Versione:** 29.0.1
- **Documentazione:** <https://docs.docker.com/>

2.4.6 Docker Compose

Docker^G Compose è uno strumento che consente di definire, configurare e gestire applicazioni multi-container. Permette di gestire l'avvio, l'arresto e l'interazione tra più container^G in modo coordinato.

- **Versione:** 2.40.3
- **Documentazione:** <https://docs.docker.com/compose/>

2.5 Modelli LLM esterni

Tra i modelli LLM^G (Large Language Models) disponibili tramite l'endpoint Zucchetti, abbiamo selezionato il modello *Gemma3:4B*, in quanto ritenuto il più adeguato per le funzionalità^G AI richieste, sia in termini di velocità di risposta sia per la qualità e coerenza del contenuto generato.



3 Architettura

In questa sezione è descritta in modo dettagliato l'architettura del nostro prodotto. Verranno illustrate le nostre scelte progettuali e i componenti del nostro sistema.

3.1 Architettura logica

Il progetto^G Second Brain è strutturato in due componenti principali:

- **Backend:** responsabile^G della gestione della *logica di business* e della persistenza dei dati.
- **Frontend:** destinato alla presentazione dei dati ottenuti dal *backend*, il *frontend* consente all'utente di interagire con le funzionalità^G del progetto^G in modo intuitivo e user-friendly.

3.1.1 Backend

Nel *backend* si è scelto di utilizzare il framework^G Laravel, che permette la realizzazione di un servizio API^G REST. L'architettura del backend è suddivisa nei seguenti strati:

- **Presentation Layer:** si occupa del routing delle richieste; i controller fungono da intermediari tra il client e i servizi, gestendo il presentation layer e instradando le richieste del client ai servizi appropriati.
- **Application Layer:** gestisce la *logica di business* dell'applicazione; questo strato è responsabile^G delle operazioni e delle regole di business, garantendo che tutte le operazioni siano eseguite correttamente;
- **Data Access Layer:** strato che gestisce la persistenza dei dati, fornendo i mezzi per salvare, recuperare e manipolare i dati.

La suddivisione in questi strati facilita lo sviluppo^G, la manutenzione e la scalabilità del *backend* del progetto^G Second Brain, conformandosi all'architettura **multi-tier** a 3-tier.

3.1.1.1 Gestione note e Esportazione

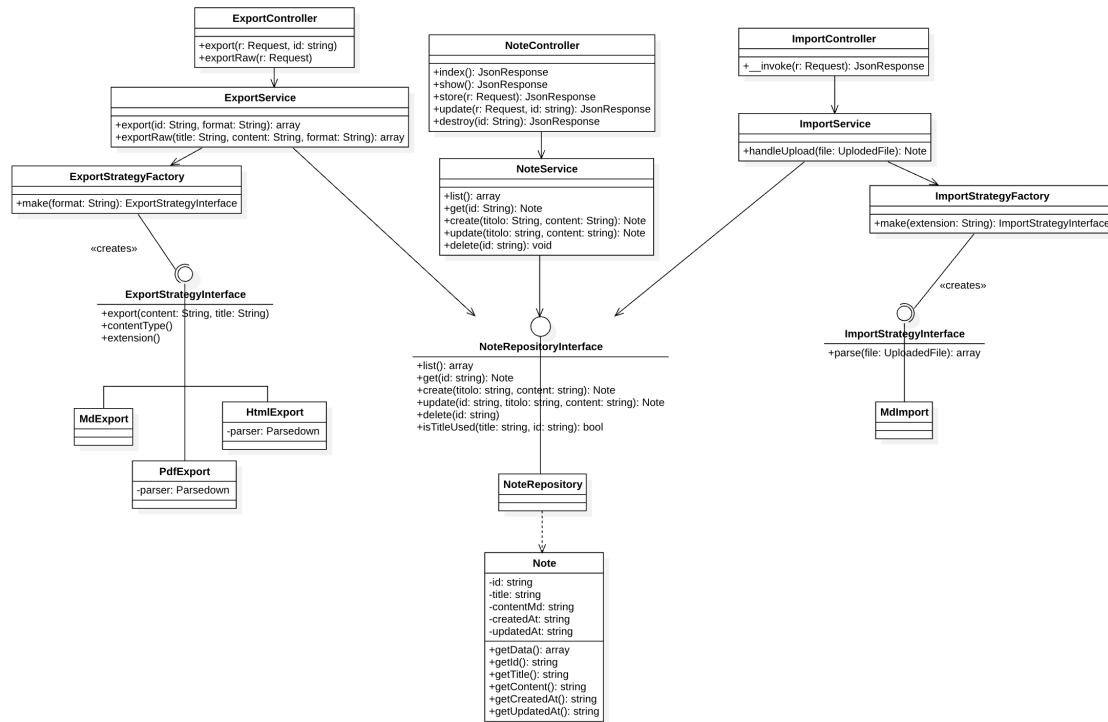


Figura 1: Diagramma delle classi per Gestione note - Esportazione - Importazione

3.1.1.1.1 NoteController È il controller HTTP responsabile^G della gestione delle richieste REST^G relative alle note. Appartiene al layer di presentazione dell'architettura layered e funge da punto di ingresso per tutte le operazioni CRUD esposte dall'API. Si limita a validare i dati in ingresso, delegare l'elaborazione a *NoteService* e restituire una risposta HTTP appropriata. Segue la descrizione dei metodi:

- **index()** : **JsonResponse** - gestisce le richieste GET `/notes`, restituendo la lista completa delle note in formato JSON con status 200, delegando il recupero dei dati a *NoteService*
- **show(string \$id)** : **JsonResponse** - gestisce le richieste GET `/notes/{id}` e restituisce i dettagli di una singola nota identificata dall'id fornito
- **store(Request \$request)** : **JsonResponse** - gestisce le richieste POST `/notes`, valida i campi, delega a *NoteService* la creazione e restituisce la nota creata con status 201
- **update(Request \$request, string \$id)** : **JsonResponse** - gestisce le richieste PUT `/notes/{id}`, valida i campi e delega l'aggiornamento a *NoteService*, passando l'id e i nuovi dati
- **destroy(string \$id)** : **JsonResponse** - gestisce le richieste DELETE `/notes/{id}`, delega l'eliminazione a *NoteService* e restituisce una risposta vuota con status 204 in caso di successo

3.1.1.1.2 NoteService È il service responsabile^G della logica di business relativa alle note. Appartiene al layer applicativo dell'architettura layered e funge da intermediario tra il controller e il repository. Dipende esclusivamente dall'interfaccia *NoteRepositoryInterface*, garantendo il rispetto del principio di inversione delle dipendenze. Segue la descrizione dei metodi:

- **list()** : **array** - restituisce la lista completa delle note, delegando direttamente al repository^G
- **get(string \$id)** : **array** - restituisce i dettagli di una singola nota tramite il suo id



- `create(string $title, string $contentMd) : array` - gestisce la creazione di una nuova nota, verificando che il titolo non sia già in uso prima di delegare al repository^G
- `update(string $id, string $title, string $contentMd) : array` - gestisce l'aggiornamento di una nota esistente; prima di delegare al repository^G verifica^G che il titolo non sia già in uso, escludendo dal confronto la nota corrente così che una nota possa essere salvata con il proprio titolo invariato senza incorrere in un falso positivo
- `delete(string $id) : void` - delega l'eliminazione della nota al repository^G

3.1.1.1.3 NoteRepositoryInterface Interfaccia che definisce il contratto che ogni implementazione del repository^G di note deve rispettare. Appartiene al layer di accesso ai dati dell'architettura layered. Dichiarando la dipendenza su questa interfaccia, `NoteService` può funzionare con qualsiasi implementazione senza richiedere modifiche al codice applicativo. Segue la descrizione dei metodi:

- `list() : array` - restituisce la lista completa delle note disponibili, ogni elemento dell'array contiene i metadati della nota senza il contenuto completo
- `get(string $id) : Note` - restituisce i dati completi di una singola nota identificata dall'id fornito
- `create(string $title, string $contentMd) : Note` - crea una nuova nota con il titolo e il contenuto Markdown forniti, genera un identificatore e registra il timestamp di creazione
- `update(string $id, string $title, string $contentMd) : Note` - aggiorna il titolo e il contenuto di una nota esistente assieme al timestamp di ultima modifica
- `delete(string $id) : void` - elimina definitivamente la nota identificata dall'id fornito
- `isTitleUsed(string $title, ?string $excludeId = null) : bool` - verifica^G se un titolo è già utilizzato da un'altra nota; `excludeId` permette di escludere dal confronto una nota specifica, per evitare che tale nota venga bloccata dal confronto con se stessa

3.1.1.1.4 NoteRepository È l'implementazione concreta di `NoteRepositoryInterface` che persiste le note come file Markdown sul filesystem locale tramite Laravel Storage. Realizza tutti i metodi dichiarati nell'interfaccia: `list()`, `get()`, `create()`, `update()`, `delete()`, `isTitleUsed()`; per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.5 Note È una classe Model che rappresenta l'entità Nota nel dominio dell'applicazione. Esistono due tipi di metodo:

- `getData() : array` - serializza l'oggetto in array, utile per passare i dati ai layer superiori
- getter singoli (`getId()`, `getTitle()`, ...) utili quando serve accedere ad una singola proprietà senza serializzare tutto

3.1.1.1.6 ImportController È il controller HTTP responsabile^G della gestione delle richieste di importazione. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta ad una sola operazione. Si limita a validare il file, delegare l'importazione a `ImportService` e costruire la risposta HTTP. Segue la descrizione dei metodi:

- `__invoke(Request $request) : JsonResponse` - valida il parametro (cioè il campo file della richiesta), delega l'importazione a `ImportService`, passando il file caricato; in caso di successo restituisce la nota importata in formato JSON con status 201, altrimenti ritorna una risposta JSON con status 400.

3.1.1.1.7 ImportService È il service responsabile^G della logica di importazione di una nota. Appartiene al layer applicativo dell'architettura layered. Dipende da `NoteRepositoryInterface` garantendo il disaccoppiamento dall'implementazione del layer di persistenza. Segue la descrizione dei metodi:

- `handleUpload(UploadedFile $file) : Note` - estrae l'estensione del file caricato, delega la creazione della strategy corretta a `ImportStrategyFactory` ed invoca sulla strategy il metodo per estrarre il contenuto in Markdown; successivamente aggiunge un timestamp al titolo della nota per evitare conflitti e invoca `NoteRepositoryInterface::create()`, ritornando un array.

3.1.1.1.8 ImportStrategyFactory - È la factory responsabile^G della creazione della strategy di importazione corretta in base all'estensione del file caricato. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- `make(string $extension) : ImportStrategyInterface` - metodo statico che istanzia la strategy concreta corrispondente all'estensione del file caricato.

3.1.1.1.9 ImportStrategyInterface Interfaccia che definisce il contratto che ogni strategy di importazione deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato all'importazione delle note. Segue la descrizione dei metodi:

- `parse(UploadedFile $file) : array` - estrae il contenuto in Markdown e il titolo della note dal file caricato.

3.1.1.1.10 MdImport È l'implementazione concreta di `ImportStrategyInterface` per l'importazione di file `.md`. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.11 ExportController È il controller HTTP responsabile^G della gestione delle richieste di esportazione, rendendo disponibile una nota come file scaricabile. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta ad una sola operazione. Si limita a validare il formato richiesto, delega l'esportazione a `ExportService` e costruisce la risposta HTTP con gli header appropriati per il download dei file. Segue la descrizione dei metodi:

- `export(Request $request, string $id)` - valida il parametro (cioè il formato con cui si vuole esportare la nota) e delega l'esportazione a `ExportService`, passando l'id della nota e il formato richiesto; in caso di successo restituisce una risposta binaria in modo che il browser tratti la risposta come un file da scaricare, altrimenti ritorna una risposta JSON con status 404
- `exportRaw(Request $request)` - valida i parametri (cioè il titolo, il contenuto e il formato con cui si vuole esportare la nota) e delega l'esportazione a `ExportService`, passando i diversi parametri e il formato richiesto; in caso di successo restituisce una risposta binaria in modo che il browser tratti la risposta come un file da scaricare, altrimenti ritorna una risposta JSON con status 404

3.1.1.1.12 ExportService È il service responsabile^G della logica di esportazione di una nota in un formato file. Appartiene al layer applicativo dell'architettura layered e gestisce la collaborazione tra il repository^G e la factory delle strategy di esportazione. Dipende da `NoteRepositoryInterface`, garantendo il disaccoppiamento dal layer di persistenza. Segue la descrizione dei metodi:

- `export(string $id, string $format) : array` - recupera la nota identificata dall'id tramite il repository^G, delega la creazione della strategy corretta a `ExportStrategyFactory` in base al formato richiesto ed invoca sulla strategy i diversi metodi per ottenere il contenuto del file esportato, il MIME type e l'estensione



- `exportRaw(string $title, string $content, string $format) : array` - delega la creazione della strategy corretta a *ExportStrategyFactory* in base al formato richiesto ed invoca sulla strategy i diversi metodi per ottenere il contenuto del file esportato, il MIME type e l'estensione

3.1.1.1.13 ExportStrategyFactory È la factory responsabile^G della creazione della strategy di esportazione corretta in base al formato richiesto. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- `make(string $format) : ExportStrategyInterface` - metodo statico che istanzia e restituisce la strategy concreta corrispondente al formato richiesto

3.1.1.1.14 ExportStrategyInterface Interfaccia che definisce il contratto che ogni strategy di esportazione deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato all'esportazione delle note. Segue la descrizione dei metodi:

- `export (string $content, string $title) : string` - trasforma il contenuto Markdown della nota nel formato di output specifico della strategy
- `contentType() : string` - restituisce il MIME type associato al formato di esportazione, per consentire al browser di interpretare correttamente il file ricevuto
- `extension() : string` - restituisce l'estensione del file associata al formato di esportazione, utilizzata per costruire il nome del file

3.1.1.1.15 MdExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato Markdown, è la strategy più semplice del sistema in quanto il contenuto delle note è già nativamente in formato Markdown, quindi non richiede alcuna trasformazione. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.16 HtmlExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato HTML. Utilizza la libreria *Parsedown* per convertire il contenuto Markdown della nota in markup HTML, che viene poi incorporato in un documento HTML completo e ben formato. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.1.17 PdfExport È l'implementazione concreta di *ExportStrategyInterface* per l'esportazione nel formato PDF. Prima delega al parser la trasformazione del Markdown in HTML, poi utilizza la libreria *barryvdh/laravel-dompdf* per convertire l'HTML in un documento PDF binario. Per la descrizione della firma e del contratto di ciascun metodo si rimanda alla sezione dedicata all'interfaccia.

3.1.1.2 Interazione con LLM^G

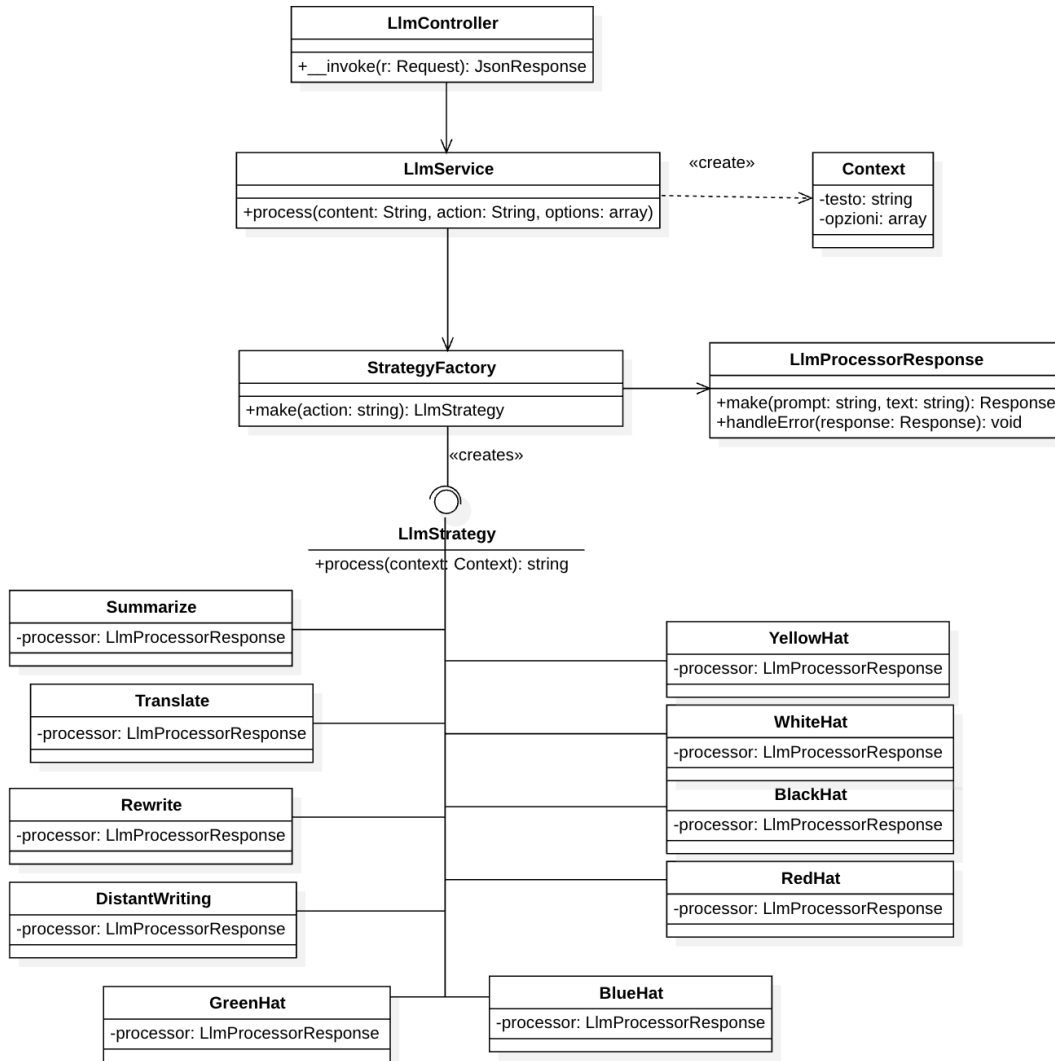


Figura 2: Diagramma delle classi per Interazione con LLM

3.1.1.2.1 LlmController È il controller HTTP responsabile^G della gestione delle richieste di elaborazione del testo tramite modelli linguistici. Appartiene al layer di presentazione dell'architettura layered ed è implementato come controller invocabile dato che la sua responsabilità è circoscritta a una sola operazione. Si limita a validare i dati in ingresso, delegare l'elaborazione a *LlmService* e restituire il risultato in formato JSON. Segue la descrizione dei metodi:

- `__invoke(Request $request, string $id) : JsonResponse` - valida i parametri (il content, l'azione da svolgere e i parametri aggiuntivi dipendenti dall'azione), delega l'elaborazione a *LlmService* e restituisce il risultato in una risposta JSON con status 200.

3.1.1.2.2 LlmService È il service responsabile^G della logica di elaborazione del testo tramite modelli linguistici. Appartiene al layer applicativo dell'architettura layered e gestisce la collaborazione tra *LlmStrategyFactory* e le strategy di elaborazione, che ne eseguono il processo. Si limita



a costruire il contesto di esecuzione e delegare il lavoro alle classi di servizio. Segue la descrizione dei metodi:

- **process(string \$content, string \$action, array \$options = []) : string** - costruisce un oggetto *LlmContext* con il testo da elaborare e le opzioni aggiuntive, delega la selezione e creazione della strategy a *LlmStrategyFactory* in base all'azione e esegue la strategy. Infine restituisce il testo prodotto dalla strategy.

3.1.1.2.3 LlmStrategyFactory È la factory responsabile^G della creazione della strategy di elaborazione testuale, elaborata in base all'azione richiesta. Appartiene al layer applicativo dell'architettura layered e implementa il pattern Factory Method, centralizzando la logica di selezione della strategy. Segue la descrizione dei metodi:

- **make(string \$action) : LlmStrategyInterface** - metodo statico che istanzia e restituisce la strategy concreta corrispondente all'azione richiesta.

3.1.1.2.4 LlmProcessorResponse Classe utility che gestisce le chiamate a un'API^G LLM^G tramite il client HTTP di Laravel. Segue la descrizione dei metodi:

- **make(string \$prompt, string \$text) : Response** - costruisce e invia una richiesta POST all'endpoint dell'API^G configurata, restituisce la Response grezza di Laravel
- **handleError(Response \$response) : void** - valida la risposta e lancia una eccezione in caso di errore, se la risposta è andata a buon fine non fa nulla

3.1.1.2.5 LlmStrategyInterface Interfaccia che definisce il contratto che ogni strategy di elaborazione testuale tramite modelli linguistici deve rispettare. Appartiene al layer applicativo dell'architettura layered e rappresenta il punto di astrazione del pattern strategy applicato alla elaborazione del testo. Segue la descrizione dei metodi:

- **process(LlmContext \$context) : string** - riceve un oggetto *LlmContext* contenente il testo da elaborare e le opzioni aggiuntive, esegue la chiamata al modello linguistico con il prompt specifico della strategy e restituisce il testo prodotto come string.

3.1.1.2.6 LlmContext È un oggetto di contesto che incapsula i dati necessari all'esecuzione di una strategy di elaborazione testuale; funge da contenitore di informazioni che vengono passata dalla *LlmService* alla strategy selezionata, evitando il passaggio di parametri multipli. Si limita a esporre i dati tramite getter.

3.1.1.2.7 Strategy concrete Tutte le strategy implementano *LlmStrategyInterface* e condividono la stessa struttura interna: costruiscono una chiamata all'API^G del modello linguistico con un prompt specifico per il tipo di richiesta di elaborazione tramite *LlmProcessorResponse* e restituiscono il contenuto generato dal modello.

- **Summarize** - elabora il testo producendo un riassunto che mantiene solo le informazioni più rilevanti, preservando la voce narrativa originale
- **Translate** - traduce il testo nella lingua specificata tramite il campo dell'array *options* del contesto
- **Rewrite** - riscrive il testo in base alle opzioni scelte dall'utente (variazione stilistica, estensione del testo, correzione grammaticale o miglioramento del lessico)
- **DistantWriting** - genera testo originale a partire da una descrizione fornita dall'utente
- **WhiteHat** - analizza il testo da un punto di vista fattuale, identificando dati oggettivi, informazioni verificabili e lacune conoscitive presenti nel contenuto
- **RedHat** - analizza il testo esprimendo le reazioni emotive e le impressioni intuitive che suscita



- **BlackHat** - individua rischi, criticità, punti deboli e potenziali conseguenze negative presenti nel testo
- **YellowHat** - identifica i punti di forza, benefici e aspetti positivi del testo
- **GreenHat** - propone idee alternative e riformulazioni possibili del contenuto
- **BlueHat** - valuta la chiarezza espositiva e il processo comunicativo del testo, fornendo una riflessione sull'organizzazione del contenuto

3.2 Architettura di deployment

La scelta della tipologia di **architettura di deployment** è stata guidata dal contesto di utilizzo del prodotto, dai requisiti di scalabilità previsti dal capitolato^G e dalle caratteristiche dello stack tecnologico. Non avendo ricevuto indicazioni specifiche riguardo a vincoli di deployment e non prevedendo significative espansioni o modifiche strutturali, il team ha optato per un'**architettura monolitica**. Questa soluzione, oltre ad essere in linea con le esigenze del prodotto, semplifica e velocizza la fase di progettazione^G e sviluppo^G ed evita la complessità introdotta da un'architettura a microservizi.

Il deployment viene gestito tramite **Docker Compose** perché, fornendo un ambiente di esecuzione preconfigurato e isolato, si semplificano notevolmente le operazioni di deployment e di manutenzione in qualsiasi ambiente, purché questo supporti Docker^G, e si evita la necessità di configurazione manuale.

3.2.1 Organizzazione dei container

Nonostante l'adozione di un'architettura monolitica dal punto di vista applicativo, il sistema è strutturato in più container^G al fine di separare le responsabilità tecniche e migliorare la gestione dell'ambiente di esecuzione.

- **app**
 - Contiene l'applicazione Laravel
 - Gestisce la business logic
 - Interagisce con il database PostgreSQL (opzionale)
- **web**
 - Utilizza nginx come web server
 - Espone l'applicazione sulla porta 8080
 - Funge da punto di ingresso per le richieste HTTP
 - Inoltra le richieste al container^G **app**
- **db**
 - Utilizza PostgreSQL
 - Responsabile^G della persistenza dei dati qualora si opti per questa soluzione invece che per la persistenza su filesystem
- **vite**(sviluppo^G)
 - Esegue un dev server per la compilazione dinamica di JavaScript e CSS
 - Espone le risorse sulla porta 5173

3.3 Architettura di dettaglio



4 Stato dei requisiti funzionali

Requisito	Descrizione	Rispettato
RF-O-1	Il sistema deve permettere all'utente di visualizzare l'archivio delle note presenti e per ogni nota il suo nome, la data di creazione e la data dell'ultima modifica	NO